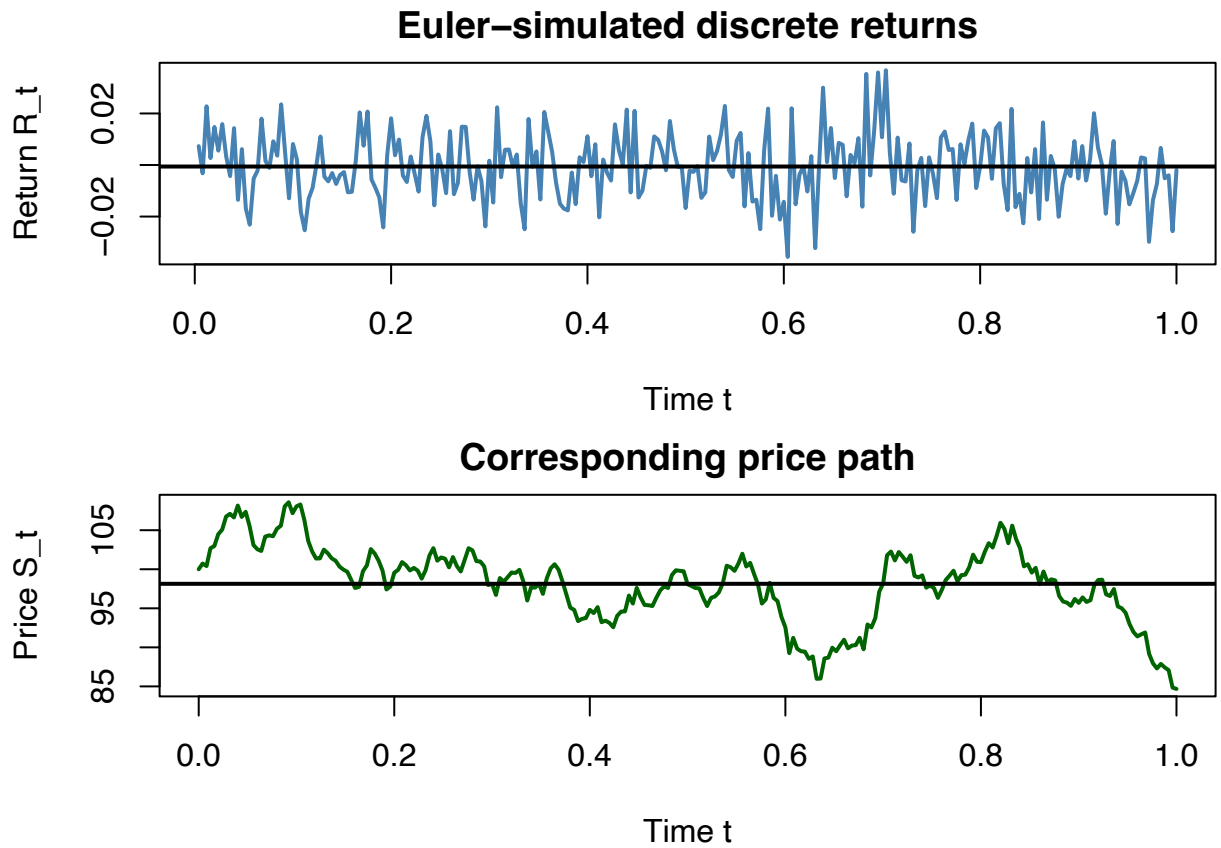
# ProblemSet1_Ex1.R

cyrilgabriele

2026-03-09

```r
# ============================================================
# Problem Set 1 - Exercise 1
# ============================================================
```

```r
S0    <- 100
mu    <- 0.08
sigma <- 0.20
T     <- 1
s     <- 250

dt    <- 1 / s
steps <- T * s
tgrid <- seq(0, T, by = dt)   # length = steps + 1

# ============================================================
# (a) One path, Euler returns + price path
# ============================================================
set.seed(13)

Z  <- rnorm(steps, mean = 0, sd = 1)     # Z_t ~ N(0,1)
dW <- sqrt(dt) * Z                       # dW_t ~ N(0, dt)
Wt <- c(0, cumsum(dW))                   # Brownian path levels W_t

Rt <- mu * dt + sigma * dW               # Euler discrete returns
St <- c(S0, S0 * cumprod(1 + Rt))        # Price path via discrete compounding

# Plot: returns (top) and prices (bottom)
par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

plot(tgrid[-1], Rt, type = "l", col = "steelblue", lwd = 2,
     xlab = "Time t", ylab = "Return R_t", main = "Euler-simulated discrete returns")
abline(h = mean(Rt), col = "black", lwd = 2)

plot(tgrid, St, type = "l", col = "darkgreen", lwd = 2,
     xlab = "Time t", ylab = "Price S_t", main = "Corresponding price path")
abline(h = mean(St), col = "black", lwd = 2)
```




```r
par(mfrow = c(1, 1))   # reset so subsequent plots are standalone

# ============================================================
# (b) mu_hat from the single path
# ============================================================
R_bar  <- mean(Rt)
mu_hat <- (1 / dt) * R_bar   # = 250 * R_bar since dt = 1/250
cat(sprintf("mu_hat      : %.6f\n", mu_hat))
```

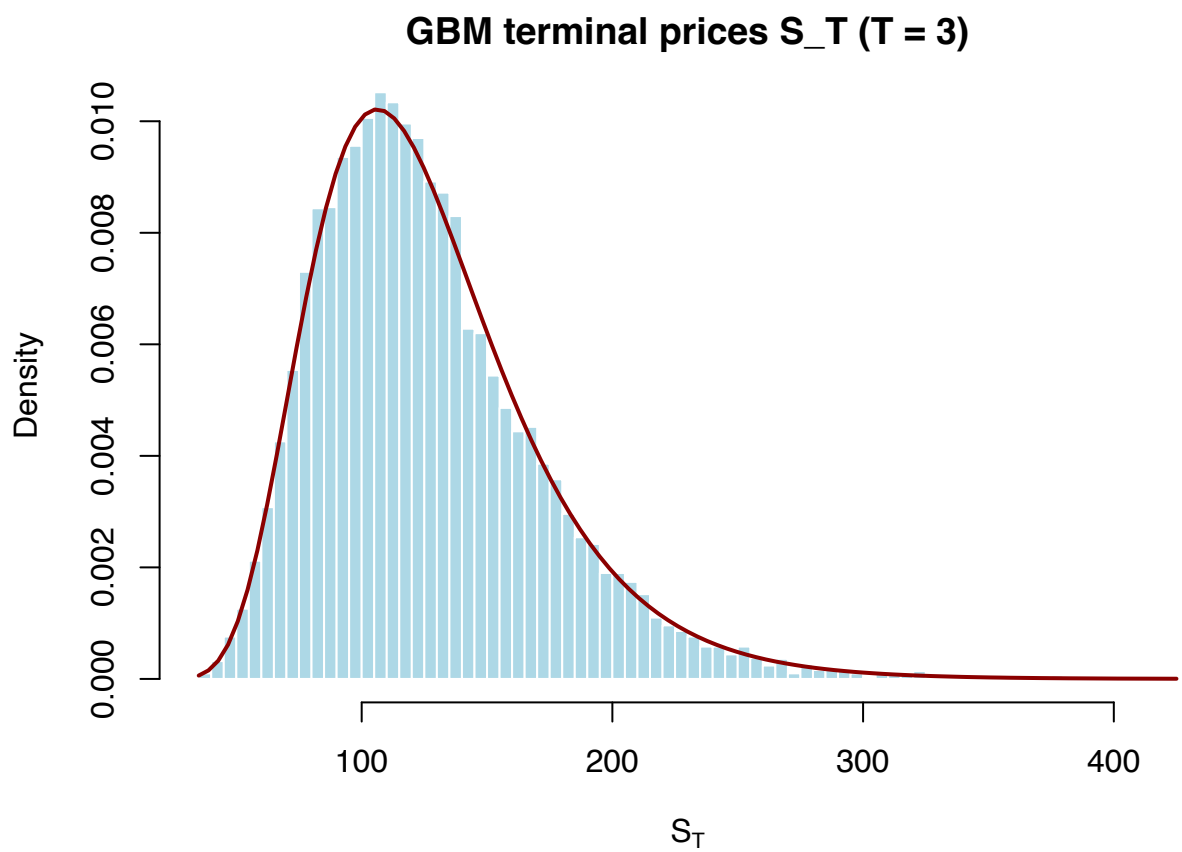
```
## mu_hat      : -0.145675
```

```r
mu_muHat_abs_diff <- abs(mu - mu_hat)
# Comment:
# mu_hat differs from mu because it is estimated from only one finite simulated
# path. Each return contains a random shock term sigma * dW_t, so the sample
# average is affected by simulation noise. Over 250 trading days, these random
# shocks do not cancel out exactly, hence the annualized estimate mu_hat is not
# exactly equal to the true drift mu. Over many independent paths, however,
# the estimate should get closer to mu.

# ============================================================
# (c) N = 10,000 independent return paths
# ============================================================
set.seed(13)   # required by the problem set
```

```r
N_paths <- 10000

Z_mat  <- matrix(rnorm(steps * N_paths, mean = 0, sd = 1), nrow = steps, ncol = N_paths)
dW_mat <- sqrt(dt) * Z_mat
R_mat  <- mu * dt + sigma * dW_mat

mu_hat_mc <- (1 / dt) * mean(R_mat)      # = 250 * mean(R_mat)
cat(sprintf("mu_hat_mc      : %.6f\n", mu_hat_mc))
```

```
## mu_hat_mc      : 0.081044
```

```r
mu_muHatmc_abs_diff     <- abs(mu - mu_hat_mc)
muHat_muHatmc_abs_diff  <- abs(mu_hat - mu_hat_mc)
# Comment:
# mu_hat_mc is much closer to the true drift mu than mu_hat from part (b),
# because it is based on averaging returns over 10,000 independent simulated
# paths instead of just one single path. With many more observations, the
# random Brownian shocks cancel out more effectively, so sampling error is
# much smaller. Hence mu_hat_mc is more stable and typically closer to mu,
# while mu_hat from one path can deviate noticeably just due to randomness.

# ============================================================
# (d) Exact solution, N = 10,000 terminal prices at T = 3
# ============================================================
set.seed(13)

S0      <- 100
mu      <- 0.08
sigma   <- 0.20
T       <- 3
N_paths <- 10000

W_T <- sqrt(T) * rnorm(N_paths, mean = 0, sd = 1)
# formula *1) (see below)
S_T <- S0 * exp((mu - 0.5 * sigma^2) * T +    # GBM exact formula
                sigma * W_T)                  # where W_t is the physical one

hist(S_T, breaks = 60, col = "lightblue", border = "white",
     freq = FALSE,
     main = "GBM terminal prices S_T (T = 3)",
     xlab = expression(S[T]))

mu_log    <- log(S0) + (mu - 0.5 * sigma^2) * T
sigma_log <- sigma * sqrt(T)
curve(dlnorm(x, meanlog = mu_log, sdlog = sigma_log), add = TRUE,
      col = "darkred", lwd = 2)
```




```r
S_T_sample_mean <- mean(S_T)
S_T_theoretical <- S0 * exp(mu * T)
cat(sprintf("E[S_T_theoretical]      : %.6f\n", S_T_theoretical))
```

```
## E[S_T_theoretical]      : 127.124915
```

```r
S_T_mean_diff      <- S_T_sample_mean - S_T_theoretical
S_T_abs_mean_diff  <- abs(S_T_mean_diff)
# Comment:
# Shape: the histogram indicates a log normal distribution.
# This since W_T is sampled from a normal dist. and then taken exponentially in
# the formula *1). This results in a log-normal dist. See also the PDF
# as a black curve with the parameters implied by the model.
# Obviously we expect a (small) difference between the analytical closed-form
# solution and the Monte Carlo approixmation approach. The abs diff of ~0.07
# is in my opinion small w.r.t to the total value of ~127.
```

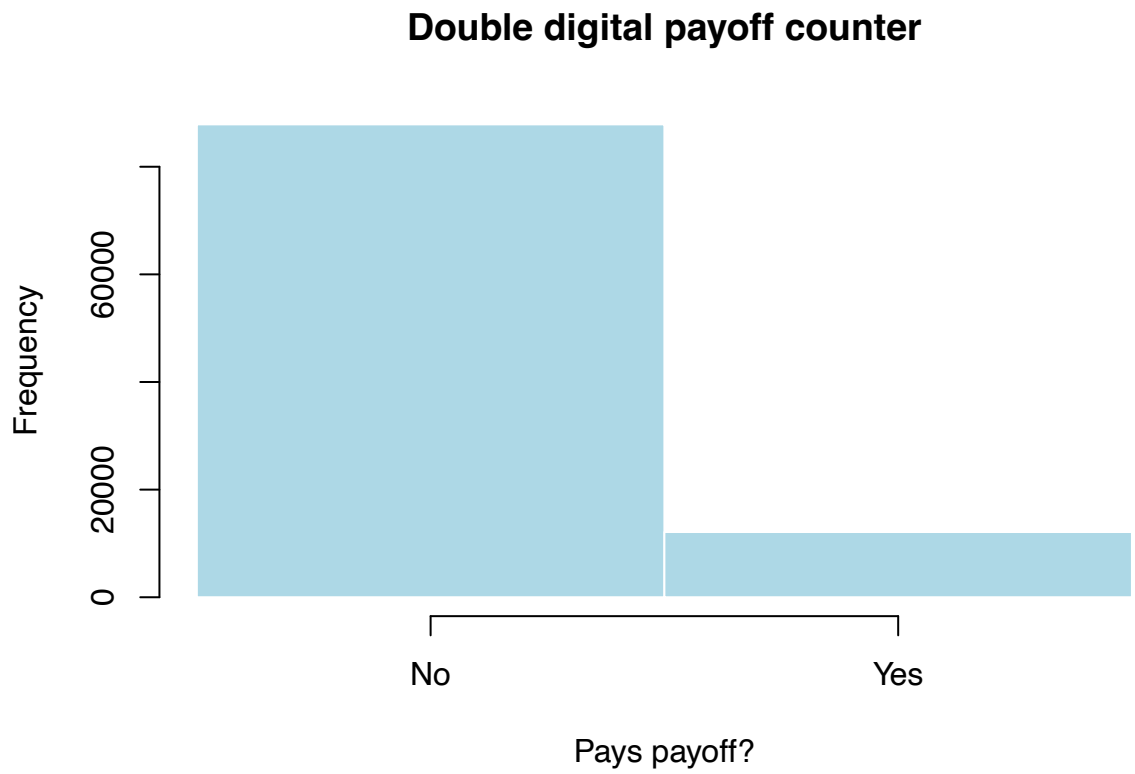
# ProblemSet1_Ex2.R

cyrilgabriele

2026-03-09

```r
# ============================================================
# Problem Set 1 - Exercise 2
# ============================================================
set.seed(13)
```

```r
S0      <- 100
sigma   <- 0.20
r       <- 0.015
T       <- 2
K1      <- 105
K2      <- 115
H       <- 20000
N_paths <- 100000

# a) simulation of S_T under Q (risk-free assumption)
Z <- rnorm(N_paths, mean=0, sd=1)
# usage here of the exact solution bc of hint
S_T <- S0 * exp((r - 0.5 * sigma^2) * T +     # GBM exact formula
                sigma * sqrt(T) * Z)

# b) payoff at maturity
payoff_indicator <- as.integer(K1 <= S_T & S_T <= K2)
D_T <- H * payoff_indicator

hist(payoff_indicator,
     breaks = seq(-0.5, 1.5, by = 1),
     col = "lightblue", border = "white",
     xaxt = "n",
     main = "Double digital payoff counter",
     xlab = "Pays payoff?")
axis(1, at = c(0, 1), labels = c("No", "Yes"))
```




```
# c) discount the expected payoff to get the price of the option today at t=0
probability_payofff = mean(payoff_indicator)
expected_payoff = mean(D_T)
V0 <- exp(-r * T) * expected_payoff
# V0 = 2352.942
cat(sprintf("V0      : %.6f\n", V0))
```

```
## V0      : 2352.942241
```

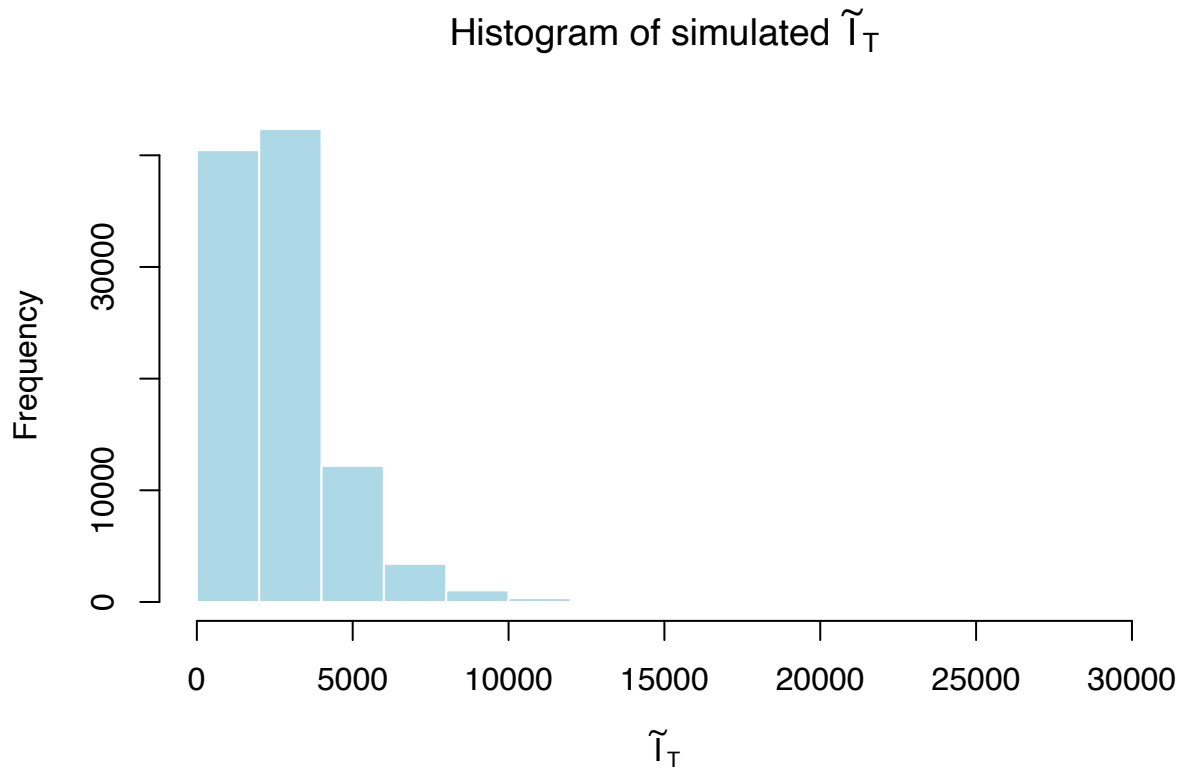
# ProblemSet1_Ex3.R

cyrilgabriele

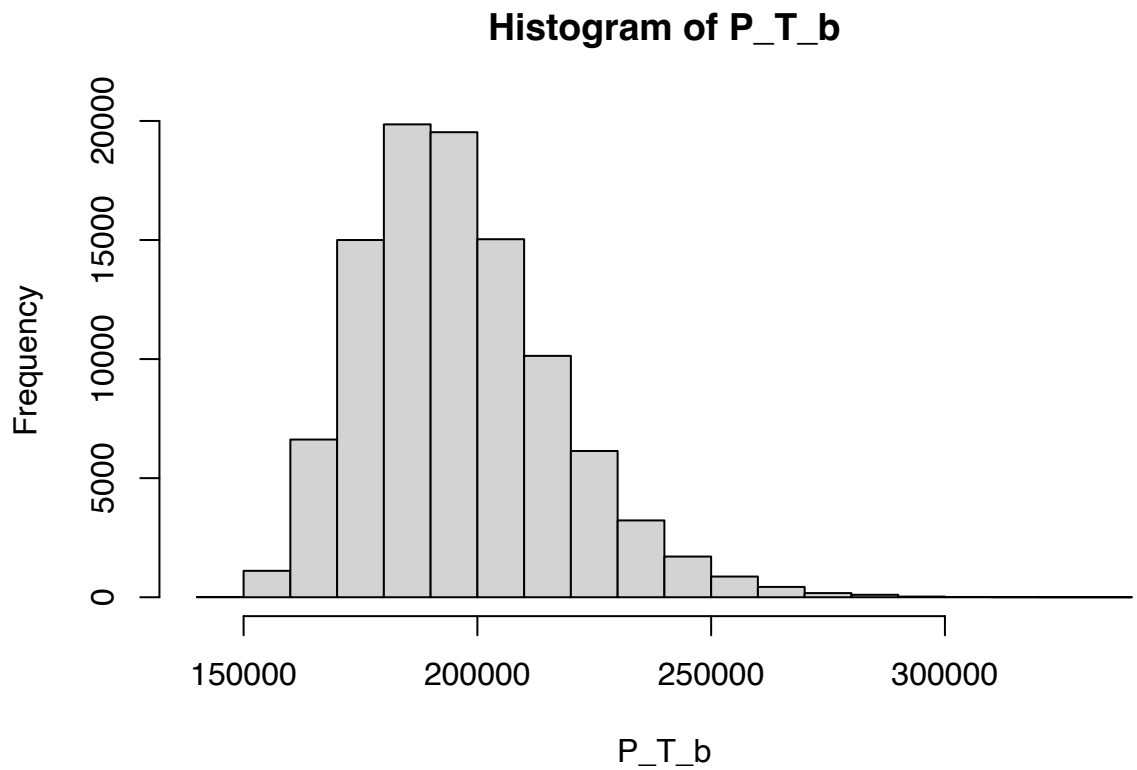
2026-03-09

```r
pi_0    <- 100000
I_0     <- 1000
mu      <- 0.05
sigma   <- 0.13
r       <- 0.01
g       <- 0.02
alpha   <- 0.30
T       <- 20
dt      <- 1
steps   <- 20
N_paths <- 100000

R_p_t <- function(I_t, I_t_minus_1, g, alpha) {
  pmax(g, alpha * ((I_t - I_t_minus_1) / I_t_minus_1))
}

P_t <- function(I_path, pi0, g, alpha) {
  R_p <- R_p_t(
    I_t         = I_path[-1],
    I_t_minus_1 = I_path[-length(I_path)],
    g           = g,
    alpha       = alpha
  )

  pi0 * prod(1 + R_p)
}

# a) simulation of I_T
set.seed(13)
I_mat_a <- matrix(NA, nrow = N_paths, ncol = steps + 1)
I_mat_a[, 1] <- I_0

for (t in 1:steps) {
  Z_t <- rnorm(N_paths)
  W_t <- sqrt(dt) * Z_t
  I_mat_a[, t + 1] <- I_mat_a[, t] * exp((mu - 0.5 * sigma^2) * dt +
                                          sigma * W_t)
}

I_T_a <- I_mat_a[, steps + 1]

hist(I_T_a,
```

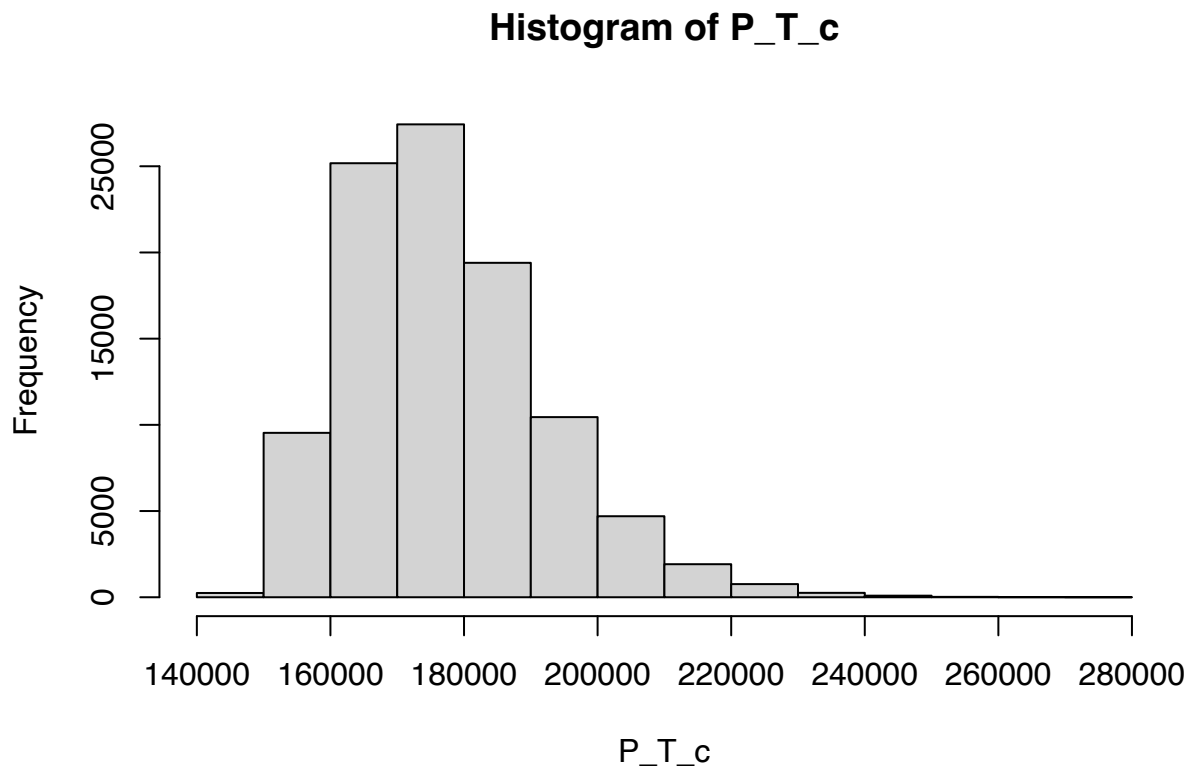
```r
     main = expression(paste("Histogram of simulated ", tilde(I)[T])),
     xlab = expression(tilde(I)[T]),
     col = "lightblue",
     border = "white")
```




```r
# ---
# b)
set.seed(13)
I_mat_b <- matrix(NA, nrow = N_paths, ncol = steps + 1)
I_mat_b[, 1] <- I_0

for (t in 1:steps) {
  Z_t <- rnorm(N_paths)
  W_t <- sqrt(dt) * Z_t
  I_mat_b[, t + 1] <- I_mat_b[, t] * exp((mu - 0.5 * sigma^2) * dt +
                                          sigma * W_t)
}

P_T_b <- apply(I_mat_b, 1, P_t, pi0 = pi_0, g = g, alpha = alpha)
hist(P_T_b)
```




```r
# ---
# c)
set.seed(13)
I_mat_c <- matrix(NA, nrow = N_paths, ncol = steps + 1)
I_mat_c[, 1] <- I_0

for (t in 1:steps) {
  Z_t <- rnorm(N_paths)
  W_t <- sqrt(dt) * Z_t
  I_mat_c[, t + 1] <- I_mat_c[, t] * exp((r - 0.5 * sigma^2) * dt +
                                          sigma * W_t)
}

P_T_c <- apply(I_mat_c, 1, P_t, pi0 = pi_0, g = g, alpha = alpha)
hist(P_T_c)
```




```r
expected_payoff <- mean(P_T_c)        # expected payoff at maturity under Q
V0 <- exp(-r * T) * expected_payoff   # product price at time 0
cat(sprintf("V0      : %.6f\n", V0))
```

```
## V0      : 145186.532411
```

```r
# Comment:
# The product price differs from the initial investment because the contract
# promises a payoff stream that is richer than risk-free compounding:
# it guarantees 2% annually while pricing discounts at only 1%, and it
# additionally offers upside participation in the index. Therefore, its
# discounted expected payoff exceeds the initial investment.
```

# ProblemSet1_Ex4.R

cyrilgabriele

2026-03-09

```r
############################################################
# Exercise 4
# Custom derivative product under GBM and risk-neutral pricing
#
# Product: Protected Target-Range Bull-Spread Note
#
# Payoff:
# X_T = G
#       + lambda * [ (S_T - K)^+ - (S_T - U)^+ ]
#       + B * 1_{L <= S_T <= U}
#
# Model under Q:
# dS_t = r S_t dt + sigma S_t dW_t^Q
#
# This script computes:
# 1) the analytical Black-Scholes price
# 2) a Monte Carlo verification under Q
#############################################################

rm(list = ls())

############################
# 1. Parameters
############################

# Market parameters
S0    <- 100      # initial stock price
r     <- 0.02     # risk-free rate
sigma <- 0.22     # volatility
T     <- 2        # maturity in years

# Product parameters
G      <- 100     # guaranteed amount at maturity
lambda <- 0.70    # participation rate in the bull spread
K      <- 100     # lower strike
U      <- 130     # upper strike / cap
L      <- 95      # lower bound of target range
B      <- 6       # fixed bonus if ST in [L, U]

############################
# 2. Black-Scholes functions
############################
```

```r
# Black-Scholes call price
bs_call <- function(S0, K, r, sigma, T) {
  d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T) / (sigma * sqrt(T))
  d2 <- d1 - sigma * sqrt(T)
  S0 * pnorm(d1) - K * exp(-r * T) * pnorm(d2)
}

# Risk-neutral probability Q(ST > x)
# Under GBM with no dividends:
# Q(ST > x) = Phi(d2(x))
d2_fun <- function(S0, x, r, sigma, T) {
  (log(S0 / x) + (r - 0.5 * sigma^2) * T) / (sigma * sqrt(T))
}

# Present value of a digital corridor paying 1 if L <= ST <= U
range_digital_pv <- function(S0, L, U, r, sigma, T) {
  d2L <- d2_fun(S0, L, r, sigma, T)
  d2U <- d2_fun(S0, U, r, sigma, T)
  exp(-r * T) * (pnorm(d2L) - pnorm(d2U))
}

############################
# 3. Analytical price
############################

# Bond component
bond_pv <- G * exp(-r * T)

# Bull spread component
call_K <- bs_call(S0, K, r, sigma, T)
call_U <- bs_call(S0, U, r, sigma, T)
spread_pv <- lambda * (call_K - call_U)

# Bonus component
bonus_pv <- B * range_digital_pv(S0, L, U, r, sigma, T)

# Total analytical price
V0_analytic <- bond_pv + spread_pv + bonus_pv

############################
# 4. Output analytical results
############################

cat("========================================\n")
```

```
## ========================================
```

```r
cat("ANALYTICAL PRICING RESULTS\n")
```

```
## ANALYTICAL PRICING RESULTS
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat(sprintf("Bond component PV      : %.6f\n", bond_pv))
```

```
## Bond component PV      : 96.078944
```

```r
cat(sprintf("Call(K=%.2f)           : %.6f\n", K, call_K))
```

```
## Call(K=100.00)           : 14.179658
```

```r
cat(sprintf("Call(U=%.2f)           : %.6f\n", U, call_U))
```

```
## Call(U=130.00)           : 4.812355
```

```r
cat(sprintf("Bull spread PV         : %.6f\n", spread_pv))
```

```
## Bull spread PV         : 6.557112
```

```r
cat(sprintf("Range bonus PV         : %.6f\n", bonus_pv))
```

```
## Range bonus PV         : 2.091156
```

```r
cat(sprintf("Total analytical price : %.6f\n\n", V0_analytic))
```

```
## Total analytical price : 104.727211
```

```r
############################
# 5. Monte Carlo verification under Q
############################

set.seed(13)

n_sim <- 1000000
Z <- rnorm(n_sim)

# Exact terminal stock price under Q
ST <- S0 * exp((r - 0.5 * sigma^2) * T + sigma * sqrt(T) * Z)

# Payoff at maturity
payoff_T <- G +
  lambda * (pmax(ST - K, 0) - pmax(ST - U, 0)) +
  B * as.numeric(ST >= L & ST <= U)

# Discounted payoff
pv <- exp(-r * T) * payoff_T
```

```r
# Monte Carlo estimator and standard error
V0_mc <- mean(pv)
SE_mc <- sd(pv) / sqrt(n_sim)
CI_low <- V0_mc - 1.96 * SE_mc
CI_high <- V0_mc + 1.96 * SE_mc

############################
# 6. Output Monte Carlo results
############################

cat("========================================\n")
```

```
## ========================================
```

```r
cat("MONTE CARLO VERIFICATION\n")
```

```
## MONTE CARLO VERIFICATION
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
cat(sprintf("Monte Carlo price      : %.6f\n", V0_mc))
```

```
## Monte Carlo price      : 104.726463
```

```r
cat(sprintf("Standard error         : %.6f\n", SE_mc))
```

```
## Standard error         : 0.008957
```

```r
cat(sprintf("95%% confidence interval: [%.6f, %.6f]\n", CI_low, CI_high))
```

```
## 95% confidence interval: [104.708906, 104.744019]
```

```r
cat(sprintf("Analytical - MC diff   : %.6f\n", V0_analytic - V0_mc))
```

```
## Analytical - MC diff   : 0.000749
```

```r
cat("========================================\n")
```

```
## ========================================
```